AutoSRE: ML implementation report

Root-cause analysis for a banking chain: Frontend, API Gateway, Transaction Service, Database. Code: github.com/vaishnavi-nss/autosre-devops (branches ml-cloud-additions and ml-and-cloud-validation). Status as of 13 Nov 2026, after live run 15.

1. What the ML part is

- **Rule-based correlation engine (the decision maker).** Alertmanager groups Prometheus alerts and posts them to the engine. It picks the deepest alerting service in the dependency chain, a signature table gives the root cause, and a weighted confidence is attached. Unknown patterns escalate to a human, never a guess.
- **Learned scorer (a second opinion, not the decision maker).** A separate read-only service pulls raw metrics from Prometheus, scores each service, and says agree, disagree or no opinion on the engine's diagnosis. It runs in **shadow** mode: it is logged, it never blocks a fix.

2. How the scorer works

- Features: per service, a robust z-score of how far each metric (cpu, memory, latency, errors, workload) moved from its own earlier behaviour. A random forest ranks the culprit service; a calibrated gradient-boosting classifier names the fault; confidence is P(service) x P(fault).
- Two training sources. **Public benchmark:** RCAEval RE1 (375 cases) and RE2 (270 cases), tested leave-one-system-out. **Our own cluster:** windows recorded during live fault injection. The scorer in use now (model b565d77f) is trained on runs 3, 4, 5-early and 8: 53 trials, 127 windows. Holding out one trial at a time it names the right service 95.3% and the right service and fault 93.7% of the time, backs 100 of 104 true claims and endorses 2 of 127 wrong ones.
- Fault start: the engine sends the time the deciding alert's fault began (its start minus its own *for* delay), not the earliest start of every alert firing. Without this a stale alert made the scorer veto correct diagnoses (found in run 7). In run 9, with a stale alert deliberately left firing, the old start disagreed with the true cause in 10 of 12 probes and the new start in 0 of 12.

3. Public benchmark (RCAEval)

Suite	Replay baseline (rule-style)	Learned model
RE1 (375 cases)	28% wrong, 37% right	61% right when forced to act; 16% wrong when gated at 0.5
RE2 (270 cases)	48% wrong	63.7% right; precision 69% at gate 0.5 up to 82% at 0.9

Remaining errors are inherent confusions: cpu vs disk, delay vs loss, memory read as disk. Adding logs gave no gain. The benchmark-trained scorer did **not** transfer to our system (it disagreed with 8 of 13 correct rule decisions in the first live test), which is why the scorer was retrained on our own data.

4. Live results on our AWS cluster

The rule engine made 0 wrong actions in the 32 injected-fault trials of the first two runs. The scorer's gate bar was written down before the data (*docs/evaluation.md*, checked by *evaluation/gate_check.py*). On the retrained model alone, runs 11 to 14:

Gate bar (written before the data)	Retrained model, runs 11-14
4 sessions with the fix (need 3)	met
Judged correct decisions (need 40), first decisions (need 20)	46, 26
Vetoes of correct decisions; 95% upper bound (need 10% or less)	0 of 46; 7.7%
True claims backed (need 95% of 40 probes or more)	50 of 52
Wrong claims caught (need 95% of 40 probes or more)	59 of 62
Unknown-cause trials escalated (need 5)	5 of 5 (all from run 11)

Every probe miss was a no_opinion (typically at the first incident, before a window exists); the scorer never endorsed a wrong claim and never disagreed with a true one. **The bar is met, SECOND_OPINION_MODE stays shadow**; switching it on is a separate decision, best one cause at a time, treating no_opinion as pass-through.

5. Why live trials were lost, and the fix

About a third of live trials were discarded as contaminated: APILatencyHigh would not clear. Cause (run 14 and 15): the services' 200m CPU limits throttle them (22% of CPU periods), which puts the idle gateway p95 at 0.45-0.77 s, on the alert's 0.5 s line. With the limit at 1000m the idle p95 fell to about 0.2 s and a 3-trial run had no contaminated trial. The live_validate.py flag *--cpu-limit* applies it. The gate evidence was gathered under the 200m limits; three trials at 1000m agree with it but are not enough to pool.

6. Still open

- Decide whether to turn the gate on (shadow today), and ESCALATE_ON_AMBIGUITY (off). Both were run on and off live with two real faults (runs 16, 17): the rule costs nothing on 130 real single-fault webhooks and escalates every two-fault incident (0 of 6 trials acted, against 9 of 9 with it off); the gate alone does not stop action on the dominant fault (the scorer misses the second fault), so it is a backstop. No wrong action in either.
- A session at the higher CPU limit, if the result is to be relied on for a differently sized cluster. Why contamination worsened late in a long session (run 13) is not shown.
- Unknown-cause evidence is from run 11 only. Faults are synthetic, on a small cluster.

7. Where to find it

- correlation-engine/: rules, engine, evaluation scripts (rcaeval_eval.py, live_validate.py, gate_check.py, diagnose_latency.py). second-opinion/: the scorer service, train.py (benchmark), train_own.py (our data).
- docs/evaluation.md (full numbers and the gate bar), docs/live-validation-run*/summary.md (one per session), docs/aws-validation.md (repeating the run).